

ISBNet: a 3D Point Cloud Instance Segmentation Network with Instance-aware Sampling and Box-aware Dynamic Convolution

Tuan Duc Ngo Binh-Son Hua Khoi Nguyen

VinAI Research, Hanoi, Vietnam

{v.tuann42, v.sonhb, v.khoindm}@vinai.io

Abstract

Existing 3D instance segmentation methods are predominated by the bottom-up design – manually fine-tuned algorithm to group points into clusters followed by a refinement network. However, by relying on the quality of the clusters, these methods generate susceptible results when (1) nearby objects with the same semantic class are packed together; or (2) large objects with loosely connected regions. To address these limitations, we introduce ISBNet, a novel cluster-free method that represents instances as kernels and decodes instance masks via dynamic convolution. To efficiently generate high-recall and discriminative kernels, we propose a simple strategy named Instance-aware Farthest Point Sampling to sample candidates and leverage the local aggregation layer inspired by PointNet++ to encode candidate features. Moreover, we show that predicting and leveraging the 3D axis-aligned bounding boxes in the dynamic convolution further boosts performance. Our method set new state-of-the-art results on ScanNetV2 (55.9), S3DIS (60.8), and STPLS3D (49.2) in terms of AP and retains fast inference time (237ms per scene on ScanNetV2). The source code and trained models are available at <https://github.com/VinAIResearch/ISBNet>.

1. Introduction

3D instance segmentation (3DIS) is a core problem of deep learning in the 3D domain. Given a 3D scene represented by a point cloud, we seek to assign each point with a semantic class and a unique instance label. 3DIS is an important 3D perception task and has a wide range of applications in autonomous driving, augmented reality, and robot navigation where point cloud data can be leveraged to complement the information provided by 2D images. Compared to 2D image instance segmentation (2DIS), 3DIS is arguably harder due to much higher variations in appearance and spatial extent along with unequal distribution of point cloud, i.e., dense near object surface and sparse elsewhere. Thus, it is not trivial to apply 2DIS methods to 3DIS.

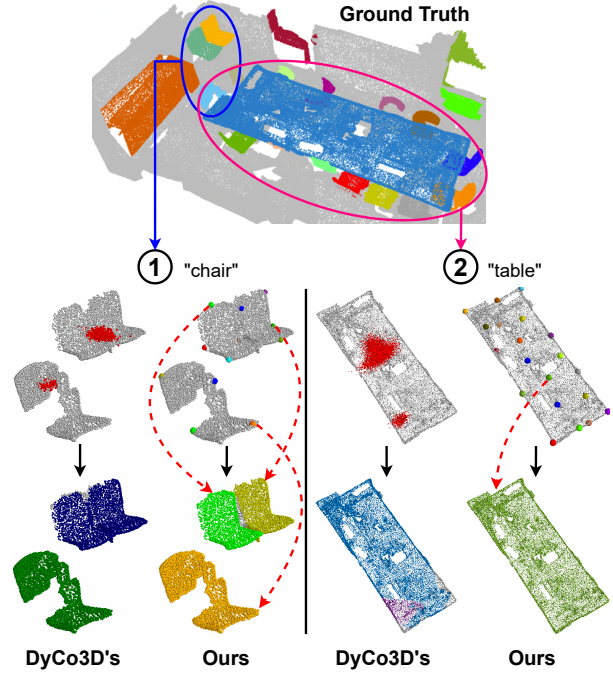


Figure 1. In DyCo3D [16], kernel prediction quality is greatly affected by the centroid-based clustering algorithm which has two issues: ① mis-grouping nearby instances and ② over-segment a large object into multiple fragments. Our method addresses these issues by *instance-aware point sampling*, achieving far better results. Each sample point aggregates information from its local context to generate a kernel for predicting its own object mask, and the final instances will be filtered and selected by an NMS.

A typical approach for 3DIS, DyCo3D [16], adopts dynamic convolution [33, 38] to predict instance masks. Specifically, points are clustered, voxelized, and passed through a 3D Unet to generate instance kernels for dynamic convolution with the feature of all points in the scene. This approach is illustrated in Fig. 2 (a). However, this approach has several limitations. First, the clustering algorithm heavily relies on the centroid-offset prediction whose quality deteriorates significantly when: (1) objects are densely packed so that two objects can be mistakenly grouped together as one object, or (2) large objects whose parts are loosely connected resulting

in different objects when clustered. These two scenarios are visualized in Fig. 1. Second, the points’ feature mostly encodes object appearance which is not distinct enough for separating different instances, especially between objects having the same semantic class.

To address the limitations of DyCo3D [16], we propose ISBNNet, a cluster-free framework for 3DIS with Instance-aware Farthest Point Sampling and Box-aware Dynamic Convolution. First, we revisit the Farthest Point Sampling (FPS) [10] and the clustering method in [5, 16, 35] and find that these algorithms generate considerably low instance recall. As a result, many objects are omitted in the subsequent stage, leading to poor performance. Motivated by this, we propose our Instance-aware Farthest Point Sampling (IA-FPS), which aims to sample query candidates in a 3D scene with high instance recall. We then introduce our Point Aggregator, incorporating the IA-FPS with a local aggregation layer to encode semantic features, shapes, and sizes of instances into instance-wise features.

Additionally, the 3D bounding box of the object is an existing supervision but has not yet been explored in the 3D instance segmentation task. Therefore, we add an auxiliary branch to our model to jointly predict the axis-aligned bounding box and the binary mask of each instance. The ground-truth axis-aligned bounding box is deduced from the existing instance mask label. Unlike Mask-DINO [25] and CondInst [33], where the auxiliary bounding box prediction is just used as a regularization of the learning process, we leverage it as an extra geometric cue in the dynamic convolution, thus further boosting the performance of the instance segmentation task.

To evaluate the performance of our approach, we conduct extensive experiments on three challenging datasets: ScanNetV2 [8], S3DIS [1], and STPLS3D [4]. ISBNNet not only achieves the highest accuracy among these three datasets, surpassing the strongest method by +2.7/2.4/3.0 on ScanNetV2, S3DIS, and STPLS3D, but also demonstrates to be highly efficient, running at 237ms per scene on ScanNetV2.

In summary, the contributions of our work are as follows:

- We propose ISBNNet, a cluster-free paradigm for 3DIS, that leverages Instance-aware Farthest Point Sampling and Point Aggregator to generate an instance feature set.
- We first introduce using the axis-aligned bounding box as an auxiliary supervision and propose the Box-aware Dynamic Convolution to decode instance binary masks.
- ISBNNet achieves state-of-the-art performance on three different datasets: ScanNetV2, S3DIS, and STPLS3D without comprehensive modifications of the model architecture and hyper-parameter tuning for each dataset.

In the following, Sec. 2 reviews prior work; Sec. 3 specifies our approach; and Sec. 4 presents our implementation details and experimental results. Sec. 5 concludes with some remarks and discussions.

2. Related Work

2D image instance segmentation (2DIS) concerns assigning each pixel in the image with one of the instance labels and semantic labels. Its approaches can be divided into three groups: proposal-based, proposal-free, and DETR-based approaches. For proposal-based methods [2, 15, 22], an object detector, e.g., Faster-RCNN [30] is leveraged to predict object bounding boxes to segment the foreground region inside detected boxes. For proposal-free methods [33, 37, 38], SOLO [37, 38] and CondInst [33] predict the instance kernels for the dynamic convolution with the feature maps to generate instance masks. For DETR-based methods [6, 7, 13, 25], Mask2Former [6] and Mask-DINO [25] employ the transformer architecture with instance queries to obtain the segmentation for each instance. Compared to 3DIS, 2DIS is arguably easier due to the structured, grid-based, and dense properties of 2D images. Hence, it is not trivial to adapt a 2DIS method to 3DIS.

3D point cloud instance segmentation (3DIS) methods are interested in labeling each point in a 3D point cloud with a semantic class and a unique instance ID. They can be categorized into proposal-based, clustering-based, and dynamic convolution-based methods. Cluster

Proposal-based methods [18, 40, 41] first detect 3D bounding boxes and then segment the foreground region inside each box to form instances. 3D-SIS [18] adapts the Mask R-CNN architecture to 3D instance segmentation and jointly learns features from two modalities of RGB images and 3D point clouds. 3D-BoNet [40] predicts a fixed number of 3D bounding boxes from a global feature vector summarizing the content of the scene and then segments the foreground points inside each box. A limitation of this approach is that the performance of instance masks heavily depends on the quality of 3D bounding boxes which is very unstable due to the huge variation and uneven distribution of 3D point cloud.

Clustering-based methods [5, 11, 16, 21, 28, 35, 36, 43] learn latent embeddings that facilitate grouping points into instances. PointGroup [21] predicts the 3D offset from each point to its instance’s centroid and obtains the clusters from two point clouds: original points and centroid-shifted points. HAIS [5] proposes a hierarchical clustering method where a small cluster can be filtered out or absorbed by a larger cluster. SoftGroup [35] proposes a soft-grouping strategy in which each point can belong to multiple clusters with different semantic classes to alleviate the semantic prediction error. One of the limitations of the clustering-based approach is that the quality of the instance masks significantly depends on the quality of the clustering, i.e., the centroid prediction, which is greatly unreliable especially when testing objects considerably differ from training objects in spatial extent.

Dynamic convolution-based methods [16, 17, 39] over-

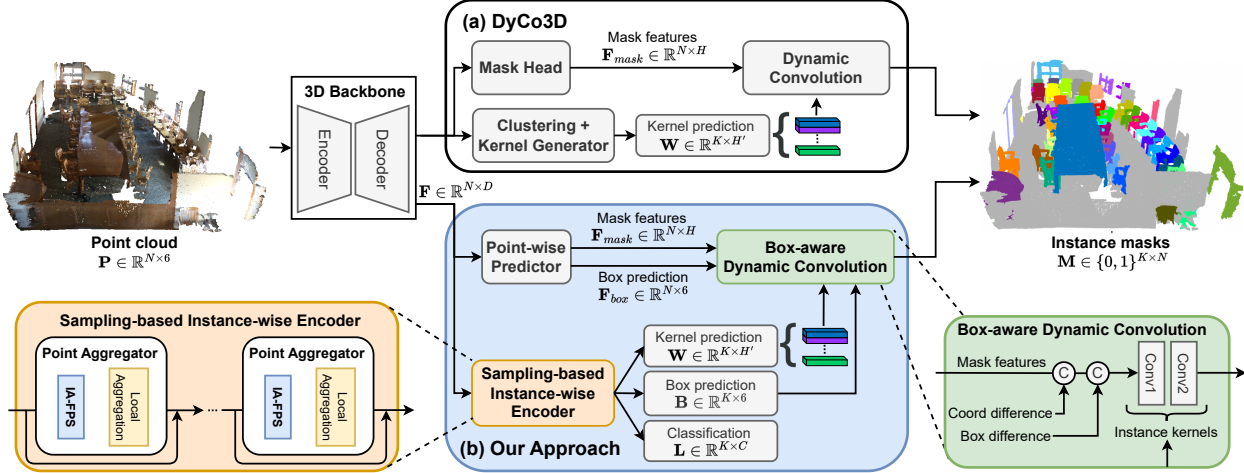


Figure 2. Overall architectures of DyCo3D [16] (block (a)) and our approach (block (b)) for 3DIS. Given a point cloud, a 3D backbone is employed to extract per-point features. For DyCo3D, it first groups points into clusters based on the predicted object centroid for each point to generate a kernel for each cluster. In the meantime, the mask head transforms the per-point features into mask features for dynamic convolution. For our approach ISBNNet, we replace the clustering algorithm with a novel sampling-based instance-wise encoder to obtain faster and more robust kernel, box, and class predictions. Furthermore, a point-wise predictor replaces the mask head of DyCo3D to output the mask and box features for a new box-aware dynamic convolution to produce more accurate instance masks.

come the limitations of proposal-based and clustering-based methods by generating kernels and then using them to convolve with the point features to generate instance masks. DyCo3D [16] adopts the clustering algorithm in [21] to generate kernels for dynamic convolution. PointInst3D [17] uses farthest-point sampling to replace the clustering in [16] in order to generate kernels. DKNet [39] introduces candidate mining and candidate aggregation to generate more discriminative instance kernels for dynamic convolution.

Our approach is a dynamic convolution-based method with two important improvements in kernel generation and dynamic convolution. Particularly, in the former, we propose a new instance encoder combining instance-aware farthest-point sampling with a point aggregation layer to generate kernels to replace clustering in DyCo3D [16]. In the latter, instead of only using the appearance feature for dynamic convolution, we additionally enhance that feature with a geometry cue namely bounding box prediction.

3. Our Approach

Problem statement: Given a 3D point cloud $\mathbf{P} \in \mathbb{R}^{N \times 6}$ where N is the number of points, and each point is represented by a 3D position and RGB color vector. We aim to segment the point cloud into K instances that are represented by a set of binary masks $\mathbf{M} \in \{0, 1\}^{K \times N}$ and a set of semantic labels $\mathbf{L} \in \mathbb{R}^{K \times C}$, where C is the number of semantic categories.

Our method consists of four main components: a 3D backbone, a point-wise predictor, a sampling-based instance-wise encoder, and a box-aware dynamic convolution. The 3D backbone takes a 3D point cloud as input to extract per-point

# sampling points	2048	512	256	128
FPS	99.3%	93.3%	85.4%	71.3%
IA-FPS	100%	98.4%	94.5%	89.2%
Clustering	75.5%	75.5%	75.5%	75.5%

Table 1. The recall of different sampling methods on ScanNetV2 validation set. FPS is the standard Farthest Point Sampling, IA-FPS is our proposed Instance-aware Farthest Point Sampling. Clustering is the algorithm used in [5, 16, 35] and its recall does not depend on the number of sampling points.

features. Our backbone network extracts feature $f^{(i)} \in \mathbb{R}^D$ where $i = 1, \dots, N$ for each point of the input point cloud. We follow previous methods [5, 35, 39] to adopt a U-Net with sparse convolutions [12] as our backbone. The point-wise predictor takes per-point features $\mathbf{F} \in \mathbb{R}^{N \times D}$ from the backbone and transforms them into point-wise semantic predictions, axis-aligned bounding box predictions $\mathbf{F}_{box} \in \mathbb{R}^{N \times 6}$, and mask features $\mathbf{F}_{mask} \in \mathbb{R}^{N \times H}$ for box-aware dynamic convolution. The sampling-based instance-wise encoder (Sec. 3.1) processes point-wise features to generate instance kernels, instance class labels, and bounding box parameters. Finally, the box-aware dynamic convolution (Sec. 3.2) gets the instance kernels and the mask features with the complementary box prediction to generate the final binary mask for each instance. An overview of our method is illustrated in Fig. 2.

3.1. Sampling-based Instance-wise Encoder

Given per-point features $\mathbf{F} \in \mathbb{R}^{N \times D}$ output from the backbone, we aim to produce instance-wise features $\mathbf{E} \in \mathbb{R}^{K \times D}$ where $K \ll N$. The instance-wise feature $\mathbf{E}$

is then used to predict the instance classification scores $\mathbf{L} \in \mathbb{R}^{K \times C}$, instance boxes $\mathbf{B} \in \mathbb{R}^{K \times 6}$, and instance kernels $\mathbf{W} \in \mathbb{R}^{K \times H'}$ where H' is decided by the sizes of the convolutional layers in dynamic convolution.

Typically, one can employ Farthest Point Sampling (FPS) [10] to sample a set of K candidates to generate instance kernels as in [17]. FPS greedily samples points in 3D coordinates by choosing the next points farthest away from the previous sampled ones using the pairwise distance. However, this sampling technique is inferior. First, there are many points belonging to the background categories among the K sampled candidates by FPS, wasting computational resources. Second, large objects dominate the number of sampled points hence no point is sampled from small objects. Third, the point-wise features cannot capture the local context to create instance kernels. We provide analysis in Tab. 1 to validate this observation. Particularly, we calculate the recall of the number of instances predicted by the kernels against the total ground truth instances. The recall value should be large as we expect good coverage of the clustered or sampled points on the ground-truth instances. However, as can be seen, previous methods have low recall which can be explained by that these methods do not consider instances for point clustering or sampling.

To address this issue, we propose a novel sampling-based instance-wise encoder that takes instances into account in the point sampling step. Inspired by the Set Abstraction in PointNet++ [29], we specify our instance encoder comprising a sequence of Point Aggregator (PA) blocks whose components are Instance-Aware FPS (IA-FPS) to sample candidate points covering as many foreground objects as possible and a local aggregation layer to capture the local context so as to enrich the candidate features individually. We visualize the PA in the orange block in Fig. 2 and detail our sampling below.

Instance-aware FPS. Our sampling strategy is to sample foreground points to maximally cover all instances regardless of their sizes. To achieve this goal, we opt for an iterative sampling technique as follows. Specifically, candidates are sampled from a set of points that are neither background nor chosen by previous sampled candidates. We use the point-wise semantic prediction to estimate the probability for each point to be background $m_{(0)}^{(i)} \in [0, 1]$. We also use the instance masks generated by previous k -th candidate $m_{(k)}^{(i)} \in [0, 1]$. The FPS is leveraged to sample points from the set of points $\mathbf{P}' \subset \mathbf{P}$:

$$\mathbf{P}' = \left\{ p^{(i)} \in \mathbf{P} \mid \min_{k=0..K'} \left(1 - m_{(k)}^{(i)} \right) > \tau \right\}, \quad (1)$$

where K' is the number of already chosen candidates and τ , is the hyper-parameter threshold.

Practically, in training, since the instance mask prediction is not good enough for guiding the instance sampling, K

candidates are sampled altogether at once from the predicted foreground mask $1 - m_{(0)}^{(i)} > \tau$. On the other hand, in testing, we iteratively sample smaller chunks $\{\kappa_1, \dots, \kappa_T\}$ one by one such that the subsequent chunks will be sampled from neither the background points nor points belonging to predicted masks of previous chunks. By doing so, the recall rate of IA-FPS improves a lot as shown in Tab. 1.

Local Aggregation Layer. For each candidate k , the local aggregation layer encodes and transforms the local context into its instance-wise features. Specifically, Ball-query is employed [29] to collect its Q local neighbors as the local features $\mathbf{F}_{local}^{(k)} \in \mathbb{R}^{Q \times D}$. Also, the relative coordinates between the candidates k and their neighbors q are computed and normalized with the neighborhood radius r to form the local coordinates, or $\mathbf{P}_{local}^{(k)} \in [-1, 1]^{Q \times 3}$. Next, we use an MLP layer to transform the local features $\mathbf{F}_{local}^{(k)}$ and the local coordinates $\mathbf{P}_{local}^{(k)}$ into the instance-wise features $e^{(k)}$ of candidate k . We also add a residual connection with the original features $f^{(k)}$ to avoid gradient vanishing. Concretely, the instance-wise feature can be computed as:

$$e^{(k)} = f^{(k)} + \max_q \left(\text{MLP} \left(\left[\mathbf{F}_{local}^{(k)}; \mathbf{P}_{local}^{(k)} \right] \right) \right), \quad (2)$$

where $[\cdot; \cdot]$ denotes the concatenation operations. From $\mathbf{E}$, a linear layer is used to predict instance classification scores $\mathbf{L}$, instance boxes $\mathbf{B}$, and instance kernels $\mathbf{W}$.

It is worth noting that, to obtain the instance-wise features $\mathbf{E}$, instead of using a single PA block, we propose a progressive way, by sequentially applying multiple PA blocks. In this way, the subsequent block will sample from the smaller number of points sampled by the previous block. Doing so has the same effect as stacking multiple convolutional layers in 2D images in order to increase the receptive field.

3.2. Box-aware Dynamic Convolution

In the dynamic convolution of [16, 17, 39], for each candidate k , the relative position of all points w.r.t k , $\mathbf{F}_{pos}^{(k)} \in \mathbb{R}^{N \times 3}$ and the point-wise mask features $\mathbf{F}_{mask} \in \mathbb{R}^{N \times H}$ are concatenated and convolved with instance kernels $w^{(k)}$ to obtain the instance binary mask $\hat{m}^{(k)} = \text{Sigmoid} \left(\text{Conv} \left(\left[\mathbf{F}_{mask}; \mathbf{F}_{pos}^{(k)} \right]; w^{(k)} \right) \right)$, where Conv is implemented as several convolutional layers. However, we would argue that only using the mask features and positions is sub-optimal. For example, points near the boundary of adjacent objects whose class is the same are indistinguishable from each other when only using the mask features and positions in 3D.

On the other hand, 3D bounding box delineates the shape and size of an object, which provides an important geometric cue for the prediction of object masks in instance segmentation. Our method uses bounding box predictions as an auxiliary task that regularizes instance segmentation

training. Particularly, for each point, we propose to regress the axis-aligned bounding box deduced from the object mask. The predicted boxes $\mathbf{F}_{box} \in \mathbb{R}^{N \times 6}$ are then used to condition the mask feature to generate kernels for the box-aware dynamic convolution (see the green block in Fig. 2). Each bounding box is parameterized by a 6D vector $f_{box}^{(i)} = (x_1, y_1, z_1, x_2, y_2, z_2)$ that represents the minimum and maximum bound of the point coordinates of an instance. It is worth noting that we choose to use axis-aligned bounding boxes because the ground-truth boxes are basically available for free as they can be easily constructed from ground-truth instance annotations.

Therefore, we propose to use the predicted boxes as an additional geometric cue in dynamic convolution, giving the name of our proposed box-aware dynamic convolution. Intuitively, two points will belong to the same object if their predicted boxes are similar. Our final instance mask $\hat{m}^{(k)} \in [0, 1]^{1 \times N}$ of the k -th candidate is obtained as:

$$\hat{m}^{(k)} = \text{Sigmoid} \left(\text{Conv} \left(\left[\mathbf{F}_{mask}; \mathbf{F}_{pos}^{(k)}; \mathbf{F}_{geo}^{(k)} \right]; w^{(k)} \right) \right). \quad (3)$$

The geometric feature $\mathbf{F}_{geo}^{(k)} \in \mathbb{R}^{N \times 6}$ can be calculated from the absolute difference of the bounding box predicted by the k -th instance candidate and N points in the input point cloud $\mathbf{P}$, or $f_{geo}^{(k,i)} = |f_{box}^{(i)} - f_{box}^{(k)}|$.

3.3. Network Training

We train our approach with the *Pointwise Loss* and *Instance-wise Loss*. The former is incurred at the point-wise prediction, i.e., the cross-entropy loss for semantic segmentation, and the L1 loss and gIoU loss [31] for bounding box regression. The latter is incurred at each instance prediction namely classification, box prediction, and mask prediction using the one-to-many matching loss proposed by [20] for 2D object detection. Specifically, the matching cost is the combination of instance classification and instance masks:

$$C(k, j) = \gamma_{mask} C_{mask}(\hat{m}^{(k)}, m^{(j)}) + C_{cls}(\hat{l}^{(k)}, l^{(j)}), \quad (4)$$

where C_{mask} is the dice loss [32] between two masks. Precisely, S predicted masks are matched to one ground-truth mask by duplicating the ground truth S times in Hungarian matching. In this way, the training convergence is much faster and the mask prediction performance is better than the one-to-one matching proposed by DETR [3]. Then the instance-wise loss incurred between the ground-truth masks and their matched predicted masks is defined as:

$$L_{inst} = L_{cls} + \lambda_{box} L_{box} + \lambda_{mask} L_{mask} + \lambda_{ms} L_{MS}, \quad (5)$$

where L_{cls} is the cross-entropy loss, L_{mask} is the combination of dice loss and BCE loss, L_{box} is the combination of L1 loss and gIoU loss, and L_{MS} is the Mask-Scoring loss [19].

4. Experiments

4.1. Experimental Setup

Datasets. We evaluate our method on three datasets: ScanNetV2 [8], S3DIS [1], and STPLS3D [4]. The *ScanNetV2* dataset consists of 1201, 312, and 100 scans with 18 object classes for training, validation, and testing, respectively. We report the evaluation results on the validation and test sets of ScanNetV2 as in the previous work. The *S3DIS* dataset contains 271 scenes from 6 areas with 13 categories. We report evaluations for both Area 5 and 6-fold cross-validation. The *STPLS3D* dataset is an aerial photogrammetry point cloud dataset from real-world and synthetic environments. It includes 25 urban scenes of a total of 6km² and 14 instance categories. Following [5, 35], we use scenes 5, 10, 15, 20, and 25 for validation and the rest for training.

Evaluation Metrics. Average precision commonly used for object detection and instance segmentation tasks is adopted, i.e., AP₅₀ and AP₂₅ are the scores with IoU thresholds of 50% and 25%, respectively, while AP is the averaged scores with IoU thresholds from 50% to 95% with a step size of 5%. Box AP means the average precision of the 3D axis-aligned bounding box prediction. Additionally, the S3DIS is also evaluated using mean coverage (mCov), mean weighed coverage (mWCov), mean precision (mPrec₅₀), and mean recall (mRec₅₀) with IoU threshold of 50%.

Implementation Details. We implement our model using PyTorch deep learning framework [27] and train it on 320 epochs with AdamW optimizer on a single V100 GPU. The batch size is set to 16. The learning rate is initialized to 0.004 and scheduled by a cosine annealing [42]. Following [35], we set the voxel size to 0.02m for ScanNetV2 and S3DIS, and to 0.3m for STPLS3D due to its sparsity and much larger scale. In training, the scenes are randomly cropped at a maximum number of 250,000 points. In testing, the whole scenes are fed into the network without cropping. We use the same backbone design as in [35], which outputs a feature map of 32 channels. A stack of two layers of PA is used in the sampling-based instance-aware encoder. τ is set to 0.5. We set the ball query radius r to 0.2 and 0.4 for these two layers and the number of neighbors $Q = 32$ for both layers. We also implement the box-aware dynamic convolution with two layers with a hidden dimension of 32. γ_{mask} is set to 5. λ_{box} , λ_{mask} , and λ_{ms} are set to 1, 5, and 1, respectively. In training, we set $S = 4$ and $K = 256$. In inference, we set $K = 384$ and use Non-Maximum-Suppression to remove redundant mask predictions with a threshold of 0.2. Following [14, 26, 39], we leverage the superpoints [23, 24] to align the final predicted masks on the ScanNetV2 dataset.

4.2. Main Results

ScanNetV2. We report the instance segmentation results on the hidden test set in Tab. 2 and the instance segmentation

Method	Venue	AP	AP ₅₀	AP ₂₅
SGPN [36]	CVPR 18	4.9	14.3	26.1
MTML [40]	ICCV 19	28.2	54.9	73.1
3D-BoNet [40]	NeurIPS 19	25.3	48.8	68.7
PointGroup [21]	CVPR 20	40.7	63.6	77.8
OccuSeg [14]	CVPR 20	44.3	67.2	74.2
DyCo3D [16]	CVPR 21	39.5	64.1	76.1
PE [42]	CVPR 21	39.6	64.5	77.6
HAIS [5]	ICCV 21	45.7	69.9	80.3
SSTNet [26]	ICCV 21	50.6	69.8	78.9
SoftGroup [35]	CVPR 22	50.4	<u>76.1</u>	86.5
RPGN [9]	ECCV 22	42.8	64.3	80.6
PointInst3D [17]	ECCV 22	43.8	-	-
Di&Co3D [43]	ECCV 22	47.7	70.0	80.2
DKNet [39]	ECCV 22	<u>53.2</u>	71.8	81.5
ISBNet	-	55.9	76.3	<u>84.5</u>

Table 2. 3D instance segmentation results on ScanNetV2 hidden test set in terms of AP scores. The best results are in **bold** and the second best ones are in underlined. Our proposed method achieves the highest AP, outperforming the previous strongest method.

and object detection results on the validation set in Tab. 3. On the hidden test set, ISBNet achieves 55.9/76.6 in AP/AP₅₀, set a new state-of-the-art performance on ScanNetV2 benchmark. On the validation set, our proposed method surpasses the second-best method with large margins, +3.7/5.5/3.6 in AP/AP₅₀/AP₂₅ and +2.6/6.5 in Box AP₅₀/Box AP₂₅.

S3DIS. Tab. 4 summarizes the results on Area 5 and 6-fold cross-validation of the S3DIS dataset. On both Area 5 and cross-validation evaluations, our proposed method overtakes the strongest method by large margins in almost metrics. On the 6-fold cross-validation evaluation, we achieve 74.9/76.8/77.1 in mCov/mWCov/mRec₅₀, with an improvement of +3.5/2.7/3.1 compared with the second-strongest method.

STPLS3D. Tab. 5 shows the results on the validation set of the STPLS3D dataset. Our method achieves the highest performance in all metrics and surpasses the second-best by +3.0/2.2 in AP/AP₅₀.

4.3. Qualitative Results

We visualize the qualitative results of our method, DyCo3D [16], and DKNet [39] on ScanNetV2 validation set in Fig. 3. As can be seen, our method successfully distinguishes nearby instances with the same semantic class. Due to the limitation of clustering, DyCo3D [16] mis-segments parts of the bookshelf (row 1) and merges nearby sofas (rows 2, 3). DKNet [39] over-segments the window in row 2, and also wrongly merges nearby sofas and table (row 3).

4.4. Ablation Study

We conduct a series of ablation studies on the validation set of the ScanNetV2 dataset to investigate ISBNet.

Method	AP	AP ₅₀	AP ₂₅	Box AP ₅₀	Box AP ₂₅
GSPN [41]	19.3	37.8	53.4	10.8	19.8
PointGroup [21]	34.8	51.7	71.3	48.9	61.5
HAIS [5]	43.5	64.4	75.6	53.1	64.3
DyCo3D [16]	40.6	61.0	-	45.3	58.9
SSTNet [26]	49.4	64.3	74.0	52.7	62.5
SoftGroup [35]	46.0	<u>67.6</u>	<u>78.9</u>	<u>59.4</u>	<u>71.6</u>
RPGN [9]	-	64.2	-	-	-
PointInst3D [17]	45.6	63.7	-	51.0	-
Di&Co3D [43]	47.7	67.2	77.2	-	-
DKNet [39]	<u>50.8</u>	66.7	76.9	59.0	67.4
ISBNet	54.5	73.1	82.5	62.0	78.1

Table 3. 3D instance segmentation and 3D object detection results on ScanNetV2 validation set.

Method	AP	AP ₅₀	mCov	mWCov	mPrec ₅₀	mRec ₅₀
SGPN [†] [41]	-	-	32.7	35.5	36.0	28.7
PointGroup [†] [21]	-	57.8	-	-	61.9	62.1
HAIS [†] [5]	-	-	64.3	66.0	71.1	65.0
SSTNet [†] [26]	42.7	59.3	-	-	65.6	64.2
SoftGroup [†] [35]	51.6	66.1	<u>66.1</u>	<u>68.0</u>	<u>73.6</u>	66.6
RPGN [†] [9]	-	-	-	-	64.0	63.0
PointInst3D [†] [17]	-	-	64.3	65.3	73.1	65.2
Di&Co3D [†] [43]	-	-	65.5	66.1	63.9	<u>67.2</u>
DKNet [†] [39]	-	-	64.7	65.6	70.8	65.3
ISBNet[†]	56.3	67.5	70.0	70.7	70.5	72.0
SGPN [‡] [41]	-	54.4	37.9	40.8	38.2	31.2
3D-BoNet [‡] [40]	-	-	-	-	65.6	47.7
PointGroup [‡] [21]	-	64.0	-	-	69.6	69.2
OccuSeg [‡] [14]	-	-	-	-	72.8	60.3
HAIS [‡] [5]	-	-	67.0	70.4	73.2	69.4
SSTNet [‡] [26]	54.1	67.8	-	-	73.5	73.4
SoftGroup [‡] [35]	54.4	68.9	69.3	71.7	75.3	69.8
RPGN [‡] [9]	-	-	-	-	84.5	70.5
PointInst3D [‡] [17]	-	-	<u>71.5</u>	<u>74.1</u>	76.4	<u>74.0</u>
DKNet [‡] [39]	-	-	70.3	72.8	75.3	71.1
ISBNet[‡]	60.8	70.5	74.9	76.8	<u>77.5</u>	77.1

Table 4. 3D instance segmentation results on S3DIS dataset. Methods marked with [†] are evaluated on Area 5, and methods marked with [‡] are evaluated on 6-fold cross-validation.

	AP	AP ₅₀	BCE	Focal	Dice	AP	AP ₅₀
PointGroup[21]	23.3	38.5	✓			53.6	72.1
HAIS[5]	35.1	46.7		✓		46.5	63.4
SoftGroup[35]	<u>46.2</u>	<u>61.8</u>		✓	✓	53.9	72.1
ISBNet	49.2	64.0	✓		✓	54.5	73.1

Table 5. 3D instance segmentation results on STPLS3D validation set.

Table 6. Ablation study on different combinations of mask losses on ScanNetV2 validation set.

The impact of different combinations of mask losses is

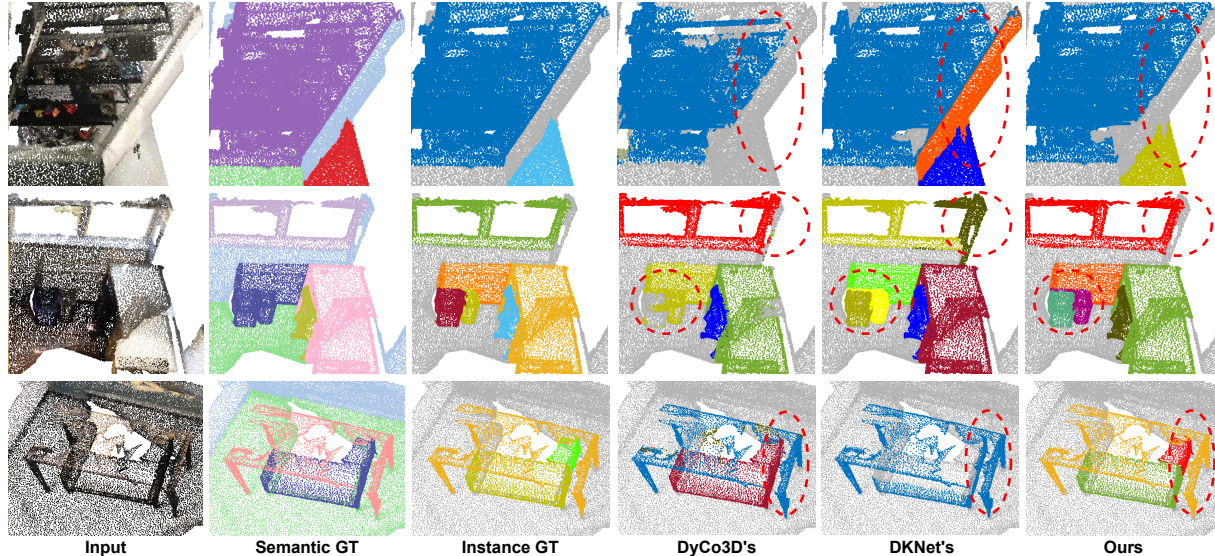


Figure 3. Representative examples on ScanNetV2 validation set. Each row shows an example with the input, Semantic ground truth, and Instance ground truth in the first three columns. Our method (the last column) produces more precise instance masks, especially in regions where multiple instances with the same semantic label lie together.

shown in Tab. 6. Notably, using a combination of binary cross entropy and dice loss yields the best result, 54.5 in AP.

The impact of each component on the overall performance is shown in Tab. 7. DyCo3D* in row 1 is our re-implementation of DyCo3D with the same backbone as [5, 35, 39], and it is trained with the one-to-many matching loss. The baseline in row 2 is a model with standard Farthest Point Sampling (FPS), standard Dynamic Convolution as in [16, 17, 39], and without Local Aggregation Layer (LAL). It can be seen that replacing the clustering and the tiny Unet in DyCo3D* decreases the performance from 49.4 to 47.9 in AP. When the standard FPS in the baseline is replaced by the Instance-aware Farthest Point Sampling (IA-FPS), the performance improves to 49.7 in row 3. When adding LAL to the baseline model, the AP score increases to 50.1 in row 4 and outperforms the AP of DyCo3D* by 0.7. Simply replacing the standard Dynamic Convolution with the Box-aware Dynamic Convolution (BA-DyCo) gains +0.7 in AP in row 5. Especially, combining the IA-FPS and PA significantly boosts the performance, +5.5/+6.8 in AP/AP₅₀ in row 6. Finally, the full approach in row 7, ISBNet achieves the best performance 54.5/73.1 in AP/AP₅₀.

The impact of axis-aligned bounding box regression is shown in Tab. 8. Without using the bounding box as an auxiliary supervision (**Aux.**), our method achieves 52.8/71.6 in AP/AP₅₀. Adding the bounding box loss during training brings a 0.6 improvement in AP. Especially, when using the bounding box as an extra geometric cue (**Geo. Cue**) in the dynamic convolution, the result significantly increases to 54.5/73.1 in AP/AP₅₀. This justifies our claim that the 3D bounding box is a critical geometric cue to distinguish

	IA-FPS	LAL	BA-DyCo	AP	AP ₅₀	AP ₂₅
DyCo3D*				49.4	67.6	77.4
Baseline				47.9	66.4	77.1
	✓			49.7	67.5	78.6
		✓		50.1	69.4	79.1
			✓	48.6	67.7	77.8
	✓	✓		53.4	71.9	81.8
ISBNet	✓	✓	✓	54.5	73.1	82.5

Table 7. Impact of each component of ISBNet on ScanNetV2 validation set. **IA-FPS**: Instance-aware Farthest Point Sampling, **LAL**: Local Aggregation Layer, **BA-DyCo**: Box-aware Dynamic Convolution. *: our improved version of DyCo3D [16].

Aux.	Geo. Cue	AP	AP ₅₀	# of PA	AP	AP ₅₀
		52.8	71.6	1	53.2	72.5
✓		53.4	71.9	2	54.5	73.1
✓	✓	54.5	73.1	3	54.3	73.0

Table 8. The impact of 3D axis-aligned bounding box regression.

Table 9. The number of Point Aggregator (PA) blocks.

instances in 3D point cloud.

The impact of the number of Point Aggregator (PA) blocks is represented in Tab. 9. With a single block of PA, our method achieves 53.2/72.5 in AP/AP₅₀. Stacking two blocks of PA gives 1.3/0.6 gains in these metrics. However, when we add more blocks, the results slightly decrease to 54.3/73.0 in AP/AP₅₀.

The impact of different designs of the Dynamic Convolution is shown in Tab. 10. Here, using two layers of dynamic

# of layers	Dimensions	# of params	AP	AP ₅₀
1	(41,1)	41	45.7	67.1
2	(25,8,1)	216	53.6	72.1
2	(41,16,1)	688	53.9	72.3
2	(41,32,1)	1376	54.5	73.1
3	(41,16,16,1)	960	53.9	72.7
3	(41,32,16,1)	1696	54.2	72.8

Table 10. Ablation on the Box-aware Dynamic Convolution.

Chunk size	Total samples K	AP	AP ₅₀	AP ₂₅
(256)	256	53.9	72.2	80.8
(384)	384	54.2	72.4	81.4
(512)	512	53.6	71.9	81.1
(128,128,128)	384	54.0	72.8	81.0
(192,128,64)	384	54.5	73.1	82.5

Table 11. Ablation on the sample chunk size of Iterative sampling.

convolution with the hidden channels of 32 gives the best results. Using only a single layer of dynamic convolution leads to a significant drop in performance. On the other hand, adding too many layers, i.e., three layers yields worse results. Reducing the number of hidden channels slightly decreases the performance. Thanks to the additional geometric cue, even with only 216 parameters of dynamic convolution, our model can achieve 53.6/72.1 in AP/AP₅₀, demonstrating the robustness of the box-aware dynamic convolution.

Ablation on the chunk size of IA-FPS. We study different designs of the sampling chunk size of IA-FPS in inference in Tab. 11. The first three rows show the results when we sample K candidates at once. Increasing the number of samples from 256 to 384 slightly improves the overall performance, but at 512 samples, the result drops to 53.6 in AP. When splitting K into smaller chunks of size (192,128,64) and sampling points based on Eq. (1), the performance further boosts to 54.5/73.1 in AP/AP₅₀ in the last row.

Runtime Analysis. Fig. 4 reports the component and total runtimes of ISBNNet and 5 recent state-of-the-art methods of 3DIS on the same Titan X GPU. All the methods can be roughly separated into three main stages: backbone, instance abstractor, and mask decoder. Our method is the fastest method, with only 237ms in total runtime and 152/53/32 ms in backbone/instance abstractor/mask decoder stages. Compared with the instance abstractors in PointGroup [21], DyCo3D [16], and SoftGroup [35] which are based on clustering, our instance abstractor based on our Point Aggregator significantly reduce the runtime. Our mask decoder, which is implemented by dynamic convolution, is the second fastest among these methods. This proves the efficiency of our proposed method.

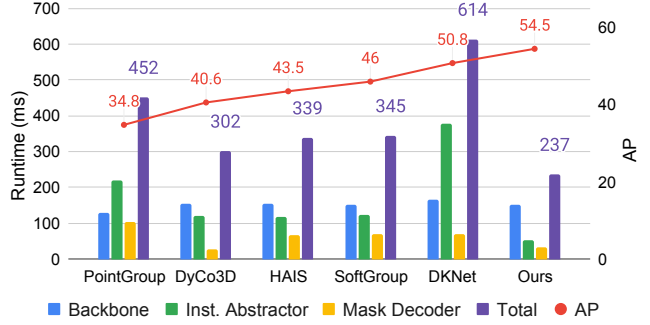


Figure 4. Components and total runtimes (in ms) and results in AP of five previous methods and ISBNNet on ScanNetV2 validation set.

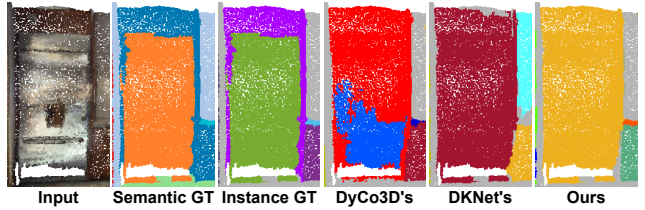


Figure 5. A hard case on ScanNetV2 validation set where a fridge is bounded by a counter. ISBNNet and previous methods wrongly merge points from these instances into a single object.

5. Conclusion

In this work, we have introduced the ISBNNet, a concise dynamic convolution-based approach to address the task of 3D point cloud instance segmentation. Considering the performance of instance segmentation models relying on the recall of candidate queries, we propose our Instance-aware Farthest Point Sampling and Point Aggregator to efficiently sample candidates in the 3D point cloud. Additionally, leveraging the 3D bounding box as auxiliary supervision and a geometric cue for dynamic convolution further enhances the accuracy of our model. Extensive experiments on ScanNetV2, S3DIS, and STPLS3D datasets show that our approach achieves robust and significant performance gain on all datasets, surpassing state-of-the-art approaches in 3D instance segmentation by large margins, i.e., +2.7, +2.4, +3.0 in AP on ScanNetV2, S3DIS and STPLS3D.

Our method is not without limitations. For example, our instance-aware FPS does not guarantee to cover all instances as it relies on the current instance prediction to make decisions for point sampling. Our proposed axis-aligned bounding box may not tightly fit the shape of complicated instances. A hard case is shown in Fig. 5 where a fridge is bounded by a counter. Our model cannot distinguish these points as they share similar bounding boxes. Addressing these limitations might lead to improvement in future work. Additionally, a new study on improving dynamic convolution by leveraging objects' geometric structures such as their shapes and sizes would be an interesting research topic.

References

- [1] Iro Armeni, Sasha Sax, Amir R Zamir, and Silvio Savarese. Joint 2d-3d-semantic data for indoor scene understanding. *arXiv preprint arXiv:1702.01105*, 2017. 2, 5
- [2] Zhaowei Cai and Nuno Vasconcelos. Cascade R-CNN: Delving into high quality object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 2
- [3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *Proceedings of the European Conference on Computer Vision*, 2020. 5
- [4] Meida Chen, Qingyong Hu, Zifan Yu, Hugues THOMAS, Andrew Feng, Yu Hou, Kyle McCullough, Fengbo Ren, and Lucio Soibelman. Stpls3d: A large-scale synthetic and real aerial photogrammetry 3d point cloud dataset. In *Proceedings of the British Machine Vision Conference*, 2022. 2, 5
- [5] Shaoyu Chen, Jiemin Fang, Qian Zhang, Wenyu Liu, and Xinggang Wang. Hierarchical aggregation for 3d instance segmentation. In *Proceedings of the International Conference on Computer Vision*, 2021. 2, 3, 5, 6, 7, 11, 12, 14
- [6] Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 2
- [7] Bowen Cheng, Alexander G. Schwing, and Alexander Kirillov. Per-pixel classification is not all you need for semantic segmentation. In *Advances in Neural Information Processing Systems*, 2021. 2
- [8] Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017. 2, 5, 11
- [9] Shichao Dong, Guosheng Lin, and Tzu-Yi Hung. Learning regional purity for instance segmentation on 3d point clouds. In *Proceedings of the European Conference on Computer Vision*, 2022. 6, 12
- [10] Yuval Eldar, Michael Lindenbaum, Moshe Porat, and Yehoshua Y Zeevi. The farthest point strategy for progressive image sampling. *IEEE Transactions on Image Processing*, 6(9):1305–1315, 1997. 2, 4
- [11] Francis Engelmann, Martin Bokeloh, Alireza Fathi, Bastian Leibe, and Matthias Nießner. 3d-mpa: Multi-proposal aggregation for 3d semantic instance segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2020. 2, 12
- [12] Benjamin Graham, Martin Engelcke, and Laurens Van Der Maaten. 3d semantic segmentation with submanifold sparse convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 3, 11
- [13] Ruohao Guo, Dantong Niu, Liao Qu, and Zhenbo Li. Sotr: Segmenting objects with transformers. In *Proceedings of the International Conference on Computer Vision*, 2021. 2
- [14] Lei Han, Tian Zheng, Lan Xu, and Lu Fang. Occuseg: Occupancy-aware 3d instance segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2020. 5, 6, 12, 14
- [15] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the International Conference on Computer Vision*, 2017. 2
- [16] Tong He, Chunhua Shen, and Anton van den Hengel. Dyco3d: Robust instance segmentation of 3d point clouds through dynamic convolution. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2021. 1, 2, 3, 4, 6, 7, 8, 12, 14
- [17] Tong He, Chunhua Shen, and Anton van den Hengel. Pointinst3d: Segmenting 3d instances by points. In *Proceedings of the European Conference on Computer Vision*, 2022. 2, 3, 4, 6, 7, 12
- [18] Ji Hou, Angela Dai, and Matthias Nießner. 3d-sis: 3d semantic instance segmentation of rgb-d scans. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019. 2, 14
- [19] Zhaojin Huang, Lichao Huang, Yongchao Gong, Chang Huang, and Xinggang Wang. Mask scoring r-cnn. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019. 5
- [20] Ding Jia, Yuhui Yuan, Haodi He, Xiaopei Wu, Haojun Yu, Weihong Lin, Lei Sun, Chao Zhang, and Han Hu. Detrs with hybrid matching. *arXiv preprint arXiv:2207.13080*, 2022. 5
- [21] Li Jiang, Hengshuang Zhao, Shaoshuai Shi, Shu Liu, Chi-Wing Fu, and Jiaya Jia. Pointgroup: Dual-set point grouping for 3d instance segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2020. 2, 3, 6, 8, 12, 14
- [22] Alexander Kirillov, Yuxin Wu, Kaiming He, and Ross Girshick. Pointrend: Image segmentation as rendering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2020. 2
- [23] Loic Landrieu and Mohamed Boussaha. Point cloud over-segmentation with graph-structured deep metric learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019. 5
- [24] Loic Landrieu and Martin Simonovsky. Large-scale point cloud semantic segmentation with superpoint graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 5
- [25] Feng Li, Hao Zhang, Shilong Liu, Lei Zhang, Lionel M Ni, Heung-Yeung Shum, et al. Mask dino: Towards a unified transformer-based framework for object detection and segmentation. *arXiv preprint arXiv:2206.02777*, 2022. 2
- [26] Zhihao Liang, Zhihao Li, Songcen Xu, Minghui Tan, and Kui Jia. Instance segmentation in 3d scenes using semantic superpoint tree networks. In *Proceedings of the International Conference on Computer Vision*, 2021. 5, 6, 12, 14
- [27] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017. 5
- [28] Quang-Hieu Pham, Thanh Nguyen, Binh-Son Hua, Gemma Roig, and Sai-Kit Yeung. Jsis3d: Joint semantic-instance segmentation of 3d point clouds with multi-task pointwise networks and multi-value conditional random fields. In *Proceedings of the IEEE Conference on Computer Vision and*

- Pattern Recognition*, 2019. 2
- [29] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, 2017. 4
 - [30] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, 2015. 2
 - [31] Hamid Rezaatoughi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019. 5
 - [32] Carole H Sudre, Wenqi Li, Tom Vercauteren, Sebastien Ourselin, and M Jorge Cardoso. Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations. In *Deep learning in medical image analysis and multimodal learning for clinical decision support*. Springer, 2017. 5
 - [33] Zhi Tian, Chunhua Shen, and Hao Chen. Conditional convolutions for instance segmentation. In *Proceedings of the European Conference on Computer Vision*. Springer, 2020. 1, 2
 - [34] Thang Vu, Kookhoi Kim, Tung M Luu, Thanh Nguyen, Junyeong Kim, and Chang D Yoo. Softgroup++: Scalable 3d instance segmentation with octree pyramid grouping. *arXiv preprint arXiv:2209.08263*, 2022. 11
 - [35] Thang Vu, Kookhoi Kim, Tung M. Luu, Xuan Thanh Nguyen, and Chang D. Yoo. Softgroup for 3d instance segmentation on 3d point clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 2, 3, 5, 6, 7, 8, 11, 12, 14
 - [36] Weiyue Wang, Ronald Yu, Qiangui Huang, and Ulrich Neumann. Sgpn: Similarity group proposal network for 3d point cloud instance segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 2, 6, 12, 14
 - [37] Xinlong Wang, Tao Kong, Chunhua Shen, Yuning Jiang, and Lei Li. Solo: Segmenting objects by locations. In *Proceedings of the European Conference on Computer Vision*. Springer, 2020. 2
 - [38] Xinlong Wang, Rufeng Zhang, Tao Kong, Lei Li, and Chunhua Shen. Solov2: Dynamic and fast instance segmentation. In *Advances in Neural Information Processing Systems*, 2020. 1, 2
 - [39] Yizheng Wu, Min Shi, Shuaiyuan Du, Hao Lu, Zhiguo Cao, and Weicai Zhong. 3d instances as 1d kernels. In *Proceedings of the European Conference on Computer Vision*, 2022. 2, 3, 4, 5, 6, 7, 11, 12, 14
 - [40] Bo Yang, Jianan Wang, Ronald Clark, Qingyong Hu, Sen Wang, Andrew Markham, and Niki Trigoni. Learning object bounding boxes for 3d instance segmentation on point clouds. In *Advances in Neural Information Processing Systems*, 2019. 2, 6, 12
 - [41] Li Yi, Wang Zhao, He Wang, Minhyuk Sung, and Leonidas J Guibas. Gspn: Generative shape proposal network for 3d instance segmentation in point cloud. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019. 2, 6
 - [42] Biao Zhang and Peter Wonka. Point cloud instance segmentation using probabilistic embeddings. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2021. 5, 6, 12
 - [43] Weiguang Zhao, Yuyao Yan, Chaolong Yang, Jianan Ye, Xi Yang, and Kaizhu Huang. Divide and conquer: 3d point cloud instance segmentation with point-wise binarization. In *Proceedings of the European Conference on Computer Vision*, 2022. 2, 6, 14

6. Supplementary Material

In this supplementary material, we provide:

- The implementation details of the MLP in Point Aggregator and the late devoxelization (Sec. 6.1).
- Performances when using a smaller backbone (Sec. 6.2).
- Per-class AP on the ScanNetV2 hidden set (Sec. 6.3).
- More qualitative results of our approach on all test datasets (Sec. 6.4).
- The run-time analysis of various methods on the ScanNetV2 validation set (Sec. 6.5).

6.1. Implementation Details

The MLP in Point Aggregator (PA). The MLP in the PA consists of three blocks of Conv-BatchNorm-ReLU. The input channel to the MLP is $D + 3$ while the hidden channel and the output channel are set to D .

Late devoxelization. Recent methods [5, 35, 39] adopt the sparse convolutional network [12] as their backbones. It requires the input point clouds to be voxelized as voxel grids and taken as input to the backbone network. The devoxelization step is performed right after the backbone to convert the voxel grids back to points. However, as all points in a voxel grid share the same features, the early devoxelization causes redundant computation and thus increases the memory consumption in later modules [34]. Therefore, following [34], we employ the late devoxelization in our model in both training and testing. Fig. 6 illustrates the differences between late and early devoxelization. The last two rows of Tab. 14 report the average inference time of our model using the early and late devoxelization, respectively. As can be seen, by late devoxelization, our model can reduce the run-time from 268 ms to 237 ms. We would note that applying the late devoxelization does not decrease the accuracy of our model.

6.2. Performances when using a smaller backbone

We report the performances of recent methods and ISB-Net using a smaller backbone as in DyCo3D on ScanNetV2 validation set in Tab 12. Our approach consistently outperforms others by a large margin on both AP/AP₅₀.

6.3. Per-class AP on the ScanNetV2 Dataset

We report the detailed results of the 18 classes on the ScanNetV2 hidden set in Tab. 13.

6.4. More Qualitative Results of Our Approach

The qualitative results of our approach on the ScanNetV2, S3DIS, and STPLS3D datasets are visualized in Fig. 7, Fig. 8, and Fig. 9, respectively.

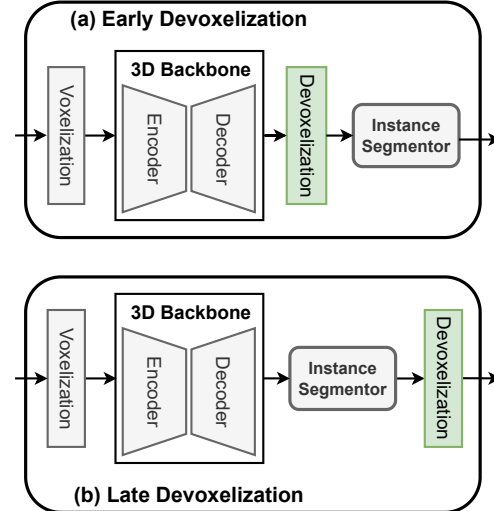


Figure 6. Difference between early and late devoxelization.

	DyCo3D-S	H AIS-S	PointInst3D-S	ISBNet-S
AP	35.4	38.0	39.6	49.5
AP₅₀	57.6	59.1	59.2	70.1

Table 12. 3DIS results of recent methods with the same small backbone as used in DyCo3D on ScanNetV2 validation set.

6.5. Run-Time Analysis

Training. The training time for our model with the default setting on the ScanNetV2 [8] training set is about 20 hours on a single NVIDIA V100 GPU.

Inference. Tab. 14 shows the average inference time of each component and the whole approach for all scans of the ScanNetV2 validation set. The first 10 rows show the run-time analysis of 10 previous methods. Row 11 presents the run-time of our proposed method. The last row reports the run-time of our model without using late devoxelization (Sec. 6.1).

Method	AP	bathub	bed	bookshe.	cabinet	chair	counter	curtain	desk	door	other	picture	fridge	s. curtain	sink	sofa	table	toilet	window
SGPN [36]	4.9	2.3	13.4	3.1	1.3	14.4	0.6	0.8	0.0	2.8	1.7	0.3	0.9	0.0	2.1	12.2	9.5	17.5	5.4
3D-BoNet [40]	25.3	51.9	32.4	25.1	13.7	34.5	3.1	41.9	6.9	16.2	13.1	5.2	20.2	33.8	14.7	30.1	30.3	65.1	17.8
3D-MPA [11]	35.5	45.7	48.4	29.9	27.7	59.1	4.7	33.2	21.2	21.7	27.8	19.3	41.3	41.0	19.5	57.4	35.2	84.9	21.3
PointGroup [21]	40.7	63.9	49.6	41.5	24.3	64.5	2.1	57.0	11.4	21.1	35.9	21.7	42.8	66.0	25.6	56.2	34.1	86.0	29.1
OccuSeg [14]	48.6	80.2	53.6	42.8	36.9	70.2	<u>20.5</u>	33.1	30.1	37.9	47.4	32.7	43.7	86.2	<u>48.5</u>	60.1	39.4	84.6	27.3
DyCo3D [16]	39.5	64.2	51.8	44.7	25.9	66.6	5.0	25.1	16.6	23.1	36.2	23.2	33.1	53.5	22.9	58.7	43.8	85.0	31.7
PE [42]	39.6	66.7	46.7	44.6	24.3	62.4	2.2	57.7	10.6	21.9	34.0	23.9	48.7	47.5	22.5	54.1	35.0	81.8	27.3
HAIS [5]	45.7	70.4	56.1	45.7	36.3	67.3	4.6	54.7	19.4	30.8	42.6	28.8	45.4	71.1	26.2	56.3	43.4	88.9	34.4
SSTNet [26]	50.6	73.8	54.9	<u>49.7</u>	31.6	69.3	17.8	37.7	19.8	33.0	46.3	57.6	51.5	<u>85.7</u>	49.4	63.7	45.7	94.3	29.0
SoftGroup [35]	50.4	66.7	57.9	37.2	<u>38.1</u>	69.4	7.2	67.7	30.3	38.7	53.1	31.9	<u>58.2</u>	75.4	31.8	64.3	49.2	90.7	38.8
RPGN [9]	42.8	63.0	50.8	36.7	24.9	65.8	1.6	<u>67.3</u>	13.1	23.4	38.3	27.0	43.4	74.8	27.4	60.9	40.6	84.2	26.7
PointInst3D [17]	43.8	<u>81.5</u>	50.7	33.8	35.5	70.3	8.9	39.0	20.8	31.3	37.3	28.8	40.1	66.6	24.2	55.3	44.2	<u>91.3</u>	29.3
DKNNet [39]	<u>53.2</u>	<u>81.5</u>	62.4	51.7	37.7	74.9	10.7	50.9	<u>30.4</u>	<u>43.7</u>	47.5	<u>58.1</u>	53.9	77.5	33.9	<u>64.0</u>	<u>50.6</u>	90.1	<u>38.5</u>
ISBNet	55.9	92.6	<u>59.7</u>	39.0	43.6	<u>72.2</u>	27.6	55.6	38.0	45.0	<u>50.5</u>	58.3	73.0	57.5	45.5	60.3	57.3	97.9	33.2

Table 13. Per-class AP of 3D instance segmentation on the ScanNetV2 hidden test set. Our proposed method achieves the highest average AP and outperforms the previous strongest method significantly.

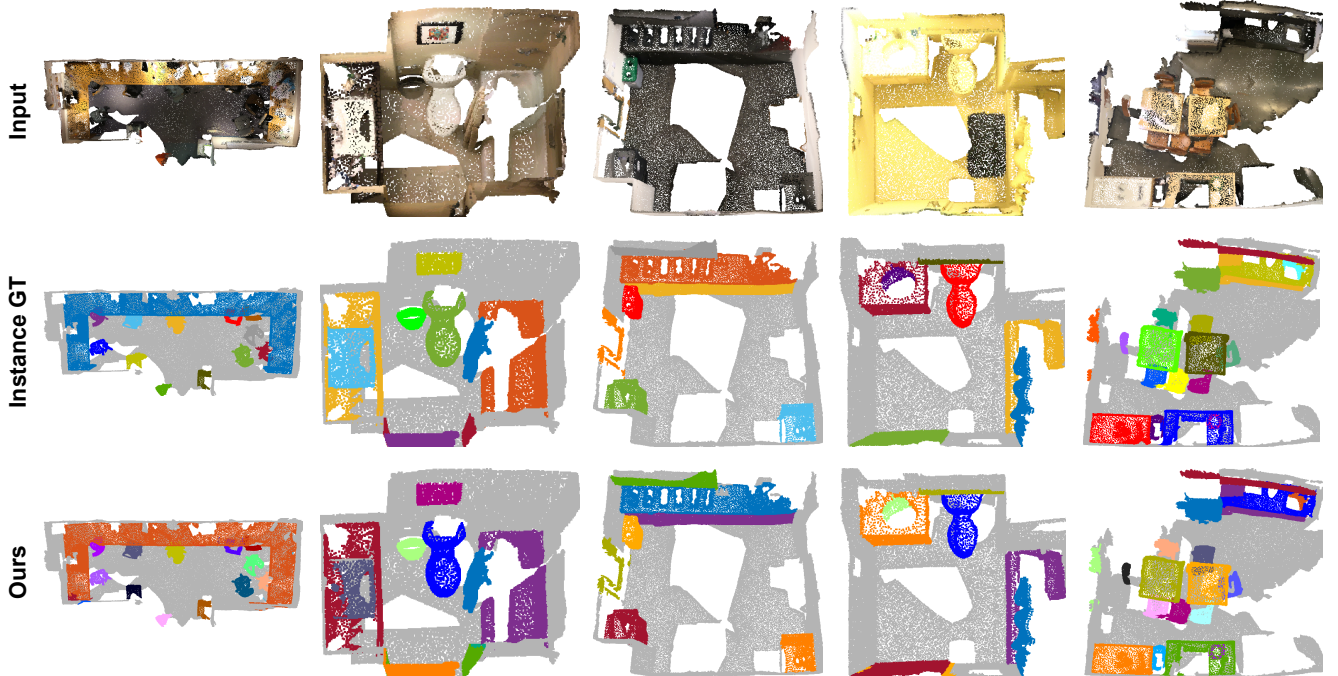


Figure 7. Qualitative results on ScanNetV2 dataset. Each column shows one example.

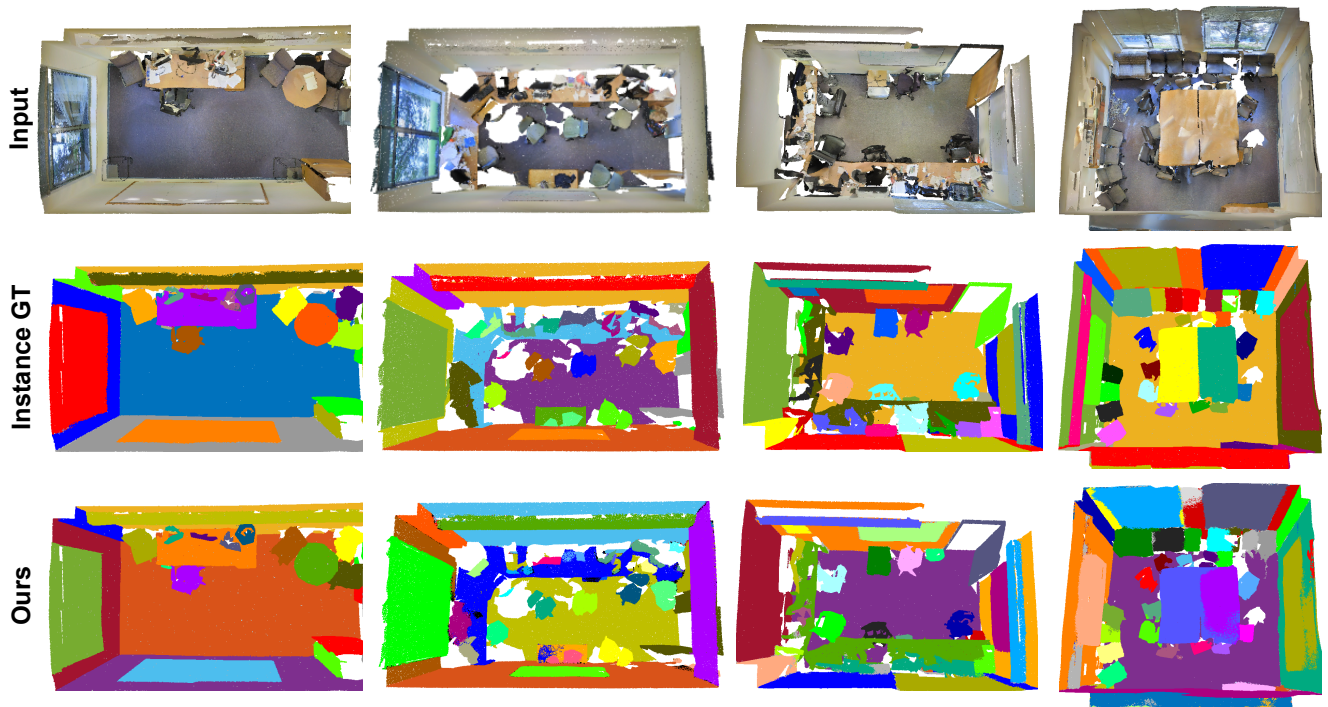


Figure 8. Qualitative results on S3DIS dataset. Each column shows one example.

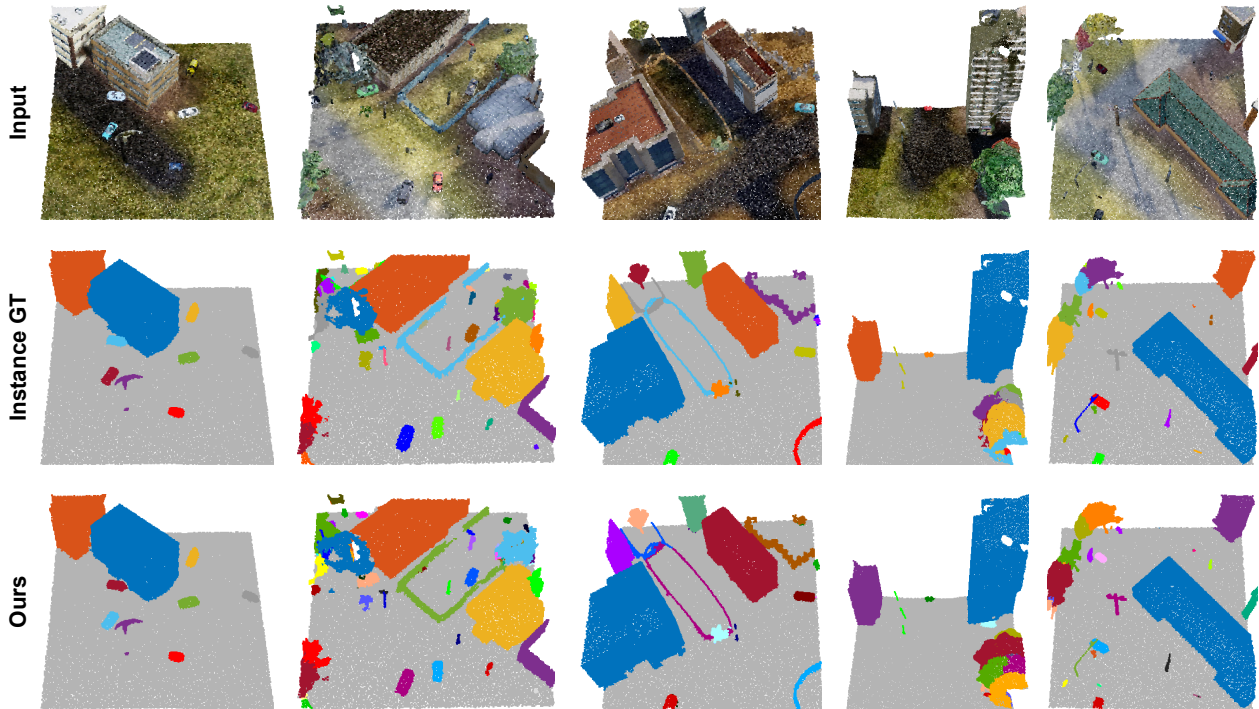


Figure 9. Qualitative results on STPLS3D dataset. Each column shows one example.

Method	Component time (ms)	Total (ms)
SGPN [36]	Backbone (GPU): 2080 Group merging (CPU): 149000 Block merging (CPU): 7119	158439
3D-BoNet [18]	Backbone (GPU): 2083 Group merging (CPU): 667 Block merging (CPU): 7119	9202
OccuSeg [14]	Backbone (GPU): 189 Group merging (CPU): 1202 Block merging (CPU): 513	1904
PointGroup [21]	Backbone (GPU): 128 Clustering (GPU+CPU): 221 ScoreNet (GPU): 103	452
SSTNet [26]	Backbone (GPU): 125 Tree net. (GPU+CPU): 229 ScoreNet (GPU): 74	428
HAIS [5]	Backbone (GPU): 154 Hier. aggre. (GPU+CPU): 118 ScoreNet (GPU): 67	339
DyCo3D [16]	Backbone (GPU): 154 Weights Gen. (GPU+CPU): 120 Dynamic Conv. (GPU): 28	302
SoftGroup [35]	Backbone (GPU): 152 Soft grouping (GPU+CPU): 123 Top-down refine. (GPU): 70	345
Di&Co [43]	Backbone (GPU): 163 Group, Vote, Merge (GPU+CPU): 275 ScoreNet (GPU): 64	502
DKNet [39]	Backbone (GPU): 165 Cand. Mining & Aggre. (GPU): 379 Dynamic Conv. + Postproc. (GPU): 70	614
ISBNet	Backbone (GPU): 152 Point Pred. & Inst. Enc. (GPU): 53 Dynamic Conv. + Postproc. (GPU): 32	237
ISBNet w/o late voxelization	Backbone (GPU): 152 Point Pred. & Inst. Enc. (GPU): 82 Dynamic Conv. + Postproc. (GPU): 34	268

Table 14. Average inference time per scan of ScanNetV2 validation set on an NVIDIA Titan X GPU.